

FLUTTER APPLICATION

ScanPro Architecture

Comprehensive technical architecture document for the ScanPro document scanning application, covering system design, data models, engine implementations, and deployment strategy.

Version 1.0

March 2026

Z.ai Engineering

CONFIDENTIAL

Table of Contents

1. Project Overview	4
2. Clean Architecture Design	4
2.1 Presentation Layer	4
2.2 Domain Layer	4
2.3 Data Layer	5
2.4 Core Layer	5
3. Complete Folder Structure	6
4. Database Schema	7
4.1 Document Model	8
4.2 Folder Model	8
4.3 Tag Model	9
4.4 OCRResult Model	9
4.5 Signature Model	9
4.6 Annotation Model	10
4.7 ScanSettings Model	10
4.8 UserProfile Model	10
4.9 SyncRecord Model	11
4.10 SearchIndex Model	11
5. Firebase Structure	11
5.1 Firestore Collections	12
5.2 Firebase Storage Paths	12
5.3 Authentication Configuration	12
5.4 Security Rules	12
6. Riverpod Setup	13
6.1 Provider Hierarchy	13
6.2 StateNotifier Pattern	13

6.3 Dependency Injection Approach	13
7. Navigation System	14
7.1 Route Configuration	14
7.2 Deep Links & Route Guards	14
8. Scanner Engine Design	15
8.1 Camera Controller	15
8.2 OpenCV Integration	15
8.3 Edge Detection Algorithm	15
8.4 Perspective Correction & Enhancement Pipeline	16
9. OCR Engine Design	16
9.1 Google ML Kit Integration	16
9.2 Text Extraction Pipeline	16
9.3 Smart Action Detection	16
10. PDF Engine Design	17
10.1 PDF Creation	17
10.2 Merge, Split & Compress Operations	17
10.3 Page Management	17
11. AI Engine Design	18
11.1 Gemini API Integration	18
12. Sync Engine Design	19
12.1 Offline-First Pattern	19
12.2 Conflict Resolution	19
12.3 Delta Sync	19
13. Security Module Design	20
13.1 Biometric Authentication	20
13.2 PIN Lock & Encryption	20
13.3 Secure Storage & Privacy Controls	20
14. UI/UX Design System	21

14.1 Material 3 Theming	21
14.2 Dark/Light Mode & Responsive Layouts	21
14.3 Animation System & Component Library	21
15. Dependencies List	22
16. Android Permissions	23
17. Build Configuration	24
17.1 build.gradle Configuration	24
17.2 Signing Configuration & Flavors	25
17.3 ProGuard Rules	25
18. MVP Roadmap	25
19. Production Roadmap	26
19.1 Phase 2 (Month 3): PDF Tools & Search	26
19.2 Phase 3 (Months 4-5): AI Features & Advanced Tools	26
19.3 Phase 4 (Month 6+): Security & Enterprise	27
20. Testing Strategy	27
20.1 Unit Testing	27
20.2 Widget Testing	27
20.3 Integration & Performance Testing	28

1. Project Overview

ScanPro is a comprehensive mobile document scanning application built with Flutter, designed to transform any smartphone into a powerful portable document scanner. The application falls under the Productivity category on both Google Play Store and Apple App Store, targeting Android as the primary platform with future iOS expansion planned. ScanPro leverages the full power of modern mobile hardware, combining advanced camera APIs, on-device machine learning, and cloud-based AI services to deliver an unparalleled document management experience.

The core goal of ScanPro is to simplify the entire document lifecycle: from capture and enhancement, through organization and search, to sharing and collaboration. Unlike basic scanner apps that merely photograph documents, ScanPro provides intelligent edge detection, perspective correction, multi-mode image enhancement, and full-text OCR that makes every scanned document instantly searchable. The application integrates with Google Gemini AI for smart features like automatic document summarization, intelligent renaming, auto-categorization, tag generation, and cross-language translation.

Target users include business professionals who need to digitize contracts and receipts on the go, students scanning lecture notes and textbook pages, legal professionals managing case documents, and healthcare workers processing patient forms. The value proposition is clear: ScanPro replaces bulky desktop scanners with a pocket-sized solution that produces professional-quality results in seconds. With Firebase-powered cloud sync, end-to-end encryption, and biometric security, users can trust that their sensitive documents are both accessible and protected across all their devices.

2. Clean Architecture Design

ScanPro follows the Clean Architecture pattern as popularized by Robert C. Martin, organized into four distinct layers with strict dependency rules. This architectural choice ensures that the business logic of the application remains independent of framework concerns, database implementations, or UI details, making the codebase highly testable, maintainable, and adaptable to changing requirements. Each layer has a single responsibility and communicates with adjacent layers through well-defined interfaces.

2.1 Presentation Layer

The Presentation layer contains all UI-related code including pages (screens), widgets (reusable UI components), and providers (Riverpod state management). This layer is responsible for rendering the user interface and handling user interactions. It depends exclusively on the Domain layer through use case interfaces, never directly accessing data sources or repositories. Riverpod providers serve as the bridge between UI and domain logic, with StateNotifiers encapsulating state transitions. The presentation layer follows the MVVM (Model-View-ViewModel) pattern where each page has a corresponding provider that manages its state and delegates business operations to domain use cases.

2.2 Domain Layer

The Domain layer is the heart of the application and contains pure business logic with zero external dependencies. It defines entities (business objects), use cases (application-specific business rules), and repository interfaces (contracts that the Data layer must implement). Every use case represents a single application action, following the Single Responsibility Principle. For example, the CaptureDocument use case encapsulates the business logic for scanning a document, while the DetectEdges use case handles the edge detection workflow. Repository interfaces are defined here but implemented in the Data layer, enabling dependency inversion. This layer is written in pure Dart with no Flutter or third-party dependencies, ensuring maximum portability.

2.3 Data Layer

The Data layer implements the repository interfaces defined in the Domain layer and manages all data operations including local storage (Hive), remote storage (Firebase), and external service integrations (Google ML Kit, Gemini API, OpenCV). It consists of models (data transfer objects), repository implementations, data sources (local and remote), and service wrappers. The Data layer follows the Repository Pattern, where each repository implementation coordinates between local and remote data sources, implementing caching strategies, offline-first patterns, and data synchronization logic. Mappers convert between external data formats and domain entities, maintaining clean boundaries.

2.4 Core Layer

The Core layer provides cross-cutting concerns shared across all other layers: constants, error handling (failures and exceptions), utility extensions, network connectivity checking, theme configuration, and reusable widgets. This layer has no business logic itself but provides the infrastructure that other layers depend on. It follows the DRY (Don't Repeat Yourself) principle, consolidating common functionality into a single source of truth. The Core layer also defines the Failure hierarchy used by the Domain layer for error propagation, ensuring consistent error handling throughout the application.

Layer	Responsibility	Key Components	Dependency Rule
Presentation	UI rendering, user interaction, state management	Pages, Widgets, Riverpod Providers	Depends on Domain only
Domain	Business logic, use cases, entity definitions	Entities, Use Cases, Repository Interfaces	No external dependencies
Data	Data access, repository implementations, external services	Models, Repositories, Data Sources, Services	Depends on Domain only
Core	Cross-cutting concerns, shared infrastructure	Constants, Errors, Extensions, Theme, Utils	No business logic

3. Complete Folder Structure

ScanPro adopts a feature-based modular folder structure that scales gracefully as the application grows. Each feature module encapsulates its own presentation, domain, and data layers, following the same Clean Architecture pattern at the module level. This approach provides strong separation of concerns, enables parallel development across teams, and makes it straightforward to extract features into separate packages if needed. The root directory separates the Flutter application code (lib/), platform-specific configurations (android/), static assets (assets/), and tests (test/).

Within the lib/ directory, the core/ folder houses shared infrastructure used across all features. The features/ directory contains 14 feature modules: home, scanner, documents, ocr, pdf_tools, search, cloud_sync, security, ai_features, signature, annotations, qr_scanner, profile, and settings. Each feature module internally mirrors the three-layer architecture with presentation/, domain/, and data/ subdirectories. The di/ (dependency injection) directory at the root level manages module registration and provider assembly, with dedicated injection modules for each major engine: scanner, OCR, PDF, sync, security, and AI.

Path	Description
scanpro/	
lib/	
main.dart	Application entry point and initialization
app.dart	Root widget with router and provider scope
core/	Shared infrastructure (constants, errors, extensions, theme, utils, widgets)
constants/ (app_constants, db_constants, firebase_constants, api_constants)	
errors/ (failures, exceptions)	
extensions/ (context_extensions, string_extensions, date_extensions)	
network/ (network_info, api_client)	
theme/ (app_theme, color_schemes, text_styles, dimensions)	Dependency injection setup and module registration
utils/ (file_utils, image_utils, date_utils, validators)	
widgets/ (loading_widget, error_widget, empty_state, custom_app_bar)	Dependency injection setup and module registration
features/	14 feature modules following Clean Architecture

home/ (presentation, domain, data)	
scanner/ (presentation/pages, domain/usecases, data/services)	
documents/ (presentation/pages, domain, data)	
ocr/ (presentation, domain, data)	
pdf_tools/ (presentation/pages, domain, data)	
search/ (presentation, domain, data)	
cloud_sync/ (presentation, domain, data)	
security/ (presentation/pages, domain, data)	
ai_features/ (presentation, domain, data)	14 feature modules following Clean Architecture
signature/ (presentation, domain, data)	
annotations/ (presentation, domain, data)	
qr_scanner/ (presentation, domain, data)	
profile/ (presentation, domain, data)	
settings/ (presentation, domain, data)	
di/	Dependency injection setup and module registration
injection.dart	
modules/ (scanner, ocr, pdf, sync, security, ai modules)	
android/ (app/build.gradle, AndroidManifest.xml, kotlin/)	Android platform configuration
assets/ (fonts/, images/, icons/)	Static resources (fonts, images, icons)
test/ (unit/, widget/, integration/)	Unit, widget, and integration test suites
pubspec.yaml	Project manifest with dependencies

4. Database Schema

ScanPro uses Hive as its primary local database, chosen for its excellent Flutter integration, fast performance with lazy loading, and native support for complex data structures. The database schema consists of 10 Hive models that cover all aspects of document management, from the core Document entity to auxiliary models for OCR results, signatures, annotations, and synchronization tracking. Each

model is annotated with Hive type adapters and field annotations for seamless serialization and deserialization. The schema is designed to support offline-first operation with efficient indexing for search operations.

4.1 Document Model

Field	Type	Description
id	String	Unique identifier (UUID v4)
title	String	Document display name
filePath	String	Original image file path
thumbnailPath	String	Thumbnail image path
pdfPath	String	Generated PDF file path
folderId	String?	Parent folder reference
tags	List	Associated tag IDs
isFavorite	bool	Favorite flag for quick access
isArchived	bool	Archive status flag
isDeleted	bool	Soft delete flag for trash
createdAt	DateTime	Creation timestamp
updatedAt	DateTime	Last modification timestamp
fileSize	int	File size in bytes
pageCount	int	Number of pages in document
ocrText	String	Extracted OCR text content
syncStatus	SyncStatus	Enum: synced, pending, conflict

4.2 Folder Model

Field	Type	Description
id	String	Unique identifier
name	String	Folder display name
parentId	String?	Parent folder for nesting
color	int	ARGB color value

icon	String	Icon identifier for display
createdAt	DateTime	Creation timestamp

4.3 Tag Model

Field	Type	Description
id	String	Unique identifier
name	String	Tag label text
color	int	ARGB color value
createdAt	DateTime	Creation timestamp

4.4 OCRResult Model

The OCRResult model stores the output of text recognition operations. It links to a Document via the documentId field and contains the full extracted text, detected language, confidence score as a percentage, and a list of detected smart actions (phone numbers, URLs, email addresses). This model enables full-text search across documents and powers the smart action detection feature that highlights actionable content within scanned text.

Field	Type	Description
id	String	Unique identifier
documentId	String	Reference to source document
text	String	Full extracted text content
language	String	Detected language code (e.g., en, zh)
confidence	double	Recognition confidence (0.0-1.0)
smartActions	List	Detected actionable items
createdAt	DateTime	Processing timestamp

4.5 Signature Model

Field	Type	Description
id	String	Unique identifier
name	String	Signature display label

imageData	Uint8List	PNG image bytes of signature
createdAt	DateTime	Creation timestamp

4.6 Annotation Model

Field	Type	Description
id	String	Unique identifier
documentId	String	Reference to document
pageIndex	int	Zero-based page index
type	AnnotationType	Enum: highlight, note, drawing, text
data	String	JSON-encoded annotation data
color	int	ARGB color value
position	String	JSON-encoded position/rect
createdAt	DateTime	Creation timestamp

4.7 ScanSettings Model

Field	Type	Description
id	String	Unique identifier (singleton)
autoCapture	bool	Auto-capture when document detected
enhancementMode	EnhancementMode	Enum: none, auto, vivid, grayscale
colorMode	ColorMode	Enum: color, grayscale, bw
quality	int	JPEG quality (70-100)
flashMode	FlashMode	Enum: auto, on, off

4.8 UserProfile Model

Field	Type	Description
id	String	Firebase UID
name	String	Display name
email	String	Email address

photoUrl	String?	Profile photo URL
isPremium	bool	Premium subscription status
syncEnabled	bool	Cloud sync preference
backupEnabled	bool	Auto-backup preference
lastSyncAt	DateTime?	Last successful sync time

4.9 SyncRecord Model

Field	Type	Description
id	String	Unique identifier
documentId	String	Reference to document
operation	SyncOperation	Enum: create, update, delete
timestamp	DateTime	Operation timestamp
status	SyncStatus	Enum: pending, completed, conflict
conflictData	String?	JSON of conflicting server data

4.10 SearchIndex Model

Field	Type	Description
id	String	Unique identifier
documentId	String	Reference to document
text	String	Indexed text fragment
fieldName	String	Source field (title, ocrText, tags)
timestamp	DateTime	Index creation timestamp

5. Firebase Structure

ScanPro leverages the full Firebase suite for cloud infrastructure, providing authentication, real-time data storage, file storage, and cloud functions. The Firebase structure follows a user-scoped architecture where all data is partitioned by user ID, ensuring complete data isolation and enabling straightforward security rules. The Firestore database uses a nested collection model under each user, while Firebase Storage organizes files by document ID with separate paths for originals, thumbnails, PDFs, and

signatures.

5.1 Firestore Collections

Collection Path	Purpose	Key Fields
users/{uid}	User profile and preferences	name, email, isPremium, syncEnabled
users/{uid}/documents/{docId}	Document metadata and sync state	title, filePath, syncStatus, updatedAt
users/{uid}/folders/{folderId}	Folder hierarchy and organization	name, parentId, color, icon
users/{uid}/tags/{tagId}	Tag definitions for categorization	name, color
users/{uid}/sync_log/{logId}	Sync operation history and audit trail	operation, timestamp, status

5.2 Firebase Storage Paths

Storage Path	Content Type	Description
users/{uid}/documents/{docId}/original.jpg	JPEG	Original scanned image at full resolution
users/{uid}/documents/{docId}/thumbnail.jpg	JPEG	Compressed thumbnail for list views
users/{uid}/documents/{docId}/pdf/{filename}.pdf	PDF	Generated PDF document file
users/{uid}/signatures/{sigId}.png	PNG	User signature image with transparency

5.3 Authentication Configuration

Firebase Authentication supports multiple sign-in methods including email/password, Google Sign-In, and anonymous authentication for trial users. The authentication flow uses Firebase Auth state listeners to automatically detect sign-in/sign-out events and update the UI accordingly. Custom auth tokens are generated for premium feature access, and the auth UID serves as the primary key for all user-scoped Firestore collections and Storage paths. Security rules enforce that users can only access their own data, preventing any cross-user data leakage.

5.4 Security Rules

Firestore security rules enforce user-scoped access: every read and write operation must authenticate the user and verify that the requested resource belongs to them. Rules validate data types and required fields on write operations, preventing malformed data from entering the database. Storage rules restrict file

uploads to authenticated users and enforce size limits (10MB for images, 50MB for PDFs). Download access is similarly restricted to the owning user. All security rules are deployed via Firebase CLI and tested with the Firestore emulator before production deployment.

6. Riverpod Setup

ScanPro uses Riverpod (flutter_riverpod: ^2.5.1) as its state management solution, chosen for its compile-time safety, testability, and decoupled architecture that doesn't depend on the widget tree. The provider hierarchy follows a layered approach that mirrors the Clean Architecture layers, with each provider having a clearly defined scope and responsibility. Riverpod's code generation capabilities (riverpod_annotation: ^2.3.5) reduce boilerplate and provide type-safe provider definitions that catch errors at compile time rather than runtime.

6.1 Provider Hierarchy

The provider hierarchy is organized into three tiers. At the foundation are infrastructure providers that create and configure singletons like Hive boxes, Firebase instances, and HTTP clients. Above these sit repository providers that implement domain repository interfaces using infrastructure dependencies. At the top are feature-specific providers (StateNotifierProviders and AsyncNotifiers) that coordinate use cases and expose reactive state to the UI. Each feature module defines its own provider scope, and cross-feature communication happens through shared repository providers rather than direct provider imports, maintaining loose coupling between features.

6.2 StateNotifier Pattern

Each feature page uses a StateNotifierProvider with a dedicated state class. The state class is immutable and uses the copyWith pattern for state transitions. For example, the ScannerNotifier manages ScannerState which includes the current scan mode, captured images list, edge detection results, and processing status. All state mutations go through the notifier's methods, which delegate to use cases and emit new states. This pattern ensures that state transitions are predictable, testable, and traceable. Async operations use AsyncValue to represent loading, data, and error states, with built-in support for pull-to-refresh and automatic error recovery.

6.3 Dependency Injection Approach

Dependency injection is handled entirely through Riverpod providers, eliminating the need for a separate DI container. The di/injection.dart file serves as the composition root where all providers are assembled. Each DI module (scanner_module.dart, ocr_module.dart, etc.) defines the providers for its feature, creating a clear dependency graph. Provider overrides enable easy testing by replacing real implementations with mocks. The ProviderScope is created in main.dart with any necessary overrides for environment-specific configurations like API keys and feature flags.

7. Navigation System

ScanPro uses GoRouter (go_router: ^14.2.0) as its declarative routing solution, providing type-safe route definitions, deep link support, and nested navigation capabilities. GoRouter was selected over Flutter's built-in Navigator 2.0 for its simpler API, built-in redirect support for authentication guarding, and first-class support for deep links and URL-based navigation. The router configuration is centralized in a single location, making it easy to understand the entire navigation graph at a glance.

7.1 Route Configuration

Route Path	Screen	Parameters
/	HomeScreen	None
/scanner	CameraScreen	None
/scanner/crop	CropScreen	imagePath
/scanner/enhance	EnhanceScreen	imagePath
/scanner/batch	BatchScanScreen	None
/documents	DocumentsScreen	folderId (optional)
/documents/:id	DocumentDetailScreen	documentId
/documents/trash	TrashScreen	None
/pdf/viewer	PdfViewerScreen	pdfPath
/pdf/merge	PdfMergeScreen	None
/pdf/split	PdfSplitScreen	pdfPath
/pdf/compress	PdfCompressScreen	pdfPath
/pdf/editor	PdfEditorScreen	pdfPath
/settings	SettingsScreen	None
/profile	ProfileScreen	None
/lock	LockScreen	None

7.2 Deep Links & Route Guards

Deep links are configured for key user flows: opening a specific document (scanpro://documents/{id}), starting a scan (scanpro://scanner), and sharing content from other apps. Android intent filters are registered in AndroidManifest.xml to handle these custom URL schemes. Route guards implement authentication checking and biometric lock verification. The GoRouter redirect function checks the

authentication state before allowing access to protected routes, redirecting unauthenticated users to the login screen. Additionally, a biometric guard intercepts navigation when the app returns from background, requiring re-authentication before resuming.

8. Scanner Engine Design

The Scanner Engine is the foundational component of ScanPro, responsible for capturing document images, detecting document boundaries, performing perspective correction, and applying image enhancement. It combines Flutter's camera plugin with OpenCV (via `opencv_dart`) for computer vision processing, delivering professional-quality scan results that rival dedicated hardware scanners. The engine is designed for real-time processing with a pipeline architecture that minimizes latency between capture and final output.

8.1 Camera Controller

The camera controller wraps the camera package (`^0.11.0`) and provides a high-level API for document scanning. It manages camera initialization, resolution configuration (favoring 4K when available), focus control with continuous auto-focus optimized for documents, and flash management. The controller exposes a real-time image stream that feeds into the edge detection pipeline. A frame rate limiter throttles processing to 15fps to balance detection responsiveness with battery life. The camera controller also handles multi-camera selection, preferring the ultra-wide lens for document capture when available, and managing camera permissions with graceful degradation.

8.2 OpenCV Integration

OpenCV integration is handled through the `opencv_dart` package (`^1.0.0`), which provides Dart bindings for core OpenCV functions via FFI. The integration focuses on image processing functions including Canny edge detection, contour finding, perspective transformation (`getPerspectiveTransform` and `warpPerspective`), adaptive thresholding, and morphological operations. All OpenCV operations run on background isolates to prevent UI jank, with results communicated back to the main isolate via ports. The OpenCV datasource abstracts the low-level FFI calls, providing a clean API for the edge detection and image processing services.

8.3 Edge Detection Algorithm

The edge detection algorithm operates in real-time on the camera feed. It begins by converting the frame to grayscale and applying Gaussian blur to reduce noise. Canny edge detection identifies edges, and contour analysis finds the largest quadrilateral contour. The algorithm uses a scoring system that considers contour area, convexity, and aspect ratio to select the most likely document boundary. Douglas-Peucker approximation simplifies the contour to four corners. A confidence score is computed based on contour quality metrics, and only contours above a threshold are displayed to the user. The auto-capture feature triggers when a high-confidence contour remains stable for 1.5 seconds.

8.4 Perspective Correction & Enhancement Pipeline

Once the document boundary is confirmed, perspective correction applies a homography transformation using the four detected corners as source points and a rectangular target as the destination. This produces a geometrically corrected document image. The enhancement pipeline then processes the corrected image through multiple stages: adaptive histogram equalization for contrast, unsharp masking for sharpness, and color correction based on the selected enhancement mode. Available modes include Auto (AI-optimized), Vivid (saturated colors for presentations), Grayscale (for text documents), and Original (no processing). The pipeline uses a chain-of-responsibility pattern, allowing each enhancement step to modify or skip processing based on the document characteristics.

9. OCR Engine Design

The OCR Engine provides on-device text recognition using Google ML Kit, enabling ScanPro to extract text from scanned documents without requiring an internet connection. The engine supports multiple languages, provides confidence scoring, and detects smart actions within recognized text. OCR results are stored locally in the OCRResult Hive model and indexed in the SearchIndex for full-text search. The OCR processing pipeline is designed to handle documents of varying quality, automatically applying pre-processing to improve recognition accuracy.

9.1 Google ML Kit Integration

ScanPro integrates three ML Kit capabilities: Text Recognition (`google_mlkit_text_recognition: ^0.12.0`) for extracting text from images, Language Identification (`google_mlkit_language_id: ^0.11.0`) for detecting the language of recognized text, and Translation (`google_mlkit_translation: ^0.11.0`) for translating text between languages. All three run entirely on-device, ensuring privacy and offline functionality. The text recognizer supports both Latin and CJK scripts with high accuracy, and the language identifier can distinguish between over 100 languages. The translation capability downloads language models on-demand and caches them for offline use.

9.2 Text Extraction Pipeline

The text extraction pipeline follows a multi-stage process. First, the scanned image undergoes pre-processing including deskewing (if not already corrected by the scanner engine), binarization for text-heavy regions, and region-of-interest detection to focus on text areas. The pre-processed image is then passed to the ML Kit text recognizer, which returns structured text blocks, lines, and elements with bounding boxes and confidence scores. Post-processing merges fragmented text blocks into coherent paragraphs, corrects common OCR errors using a dictionary-based approach, and computes an overall confidence score. The final result is stored in the OCRResult model with both the raw and processed text, enabling search indexing and smart action detection.

9.3 Smart Action Detection

Smart action detection scans the OCR output for actionable content patterns: phone numbers (using libphonenumber formatting), email addresses (RFC 5322 regex), URLs (with protocol detection), physical addresses (using address pattern matching), dates and times (natural language parsing), and monetary amounts. Each detected action includes its type, the matched text, bounding box coordinates (for highlighting in the UI), and a tap handler that launches the appropriate system action (dialer for phone numbers, mail client for emails, browser for URLs, maps for addresses). This feature transforms static scanned documents into interactive, actionable content.

10. PDF Engine Design

The PDF Engine provides comprehensive PDF manipulation capabilities using Syncfusion Flutter PDF (syncfusion_flutter_pdf: ^25.1.35) for creation and editing, and Syncfusion PDF Viewer (syncfusion_flutter_pdfviewer: ^25.1.35) for rendering. The engine supports the complete PDF lifecycle: creation from scanned images, viewing with annotation support, merging multiple PDFs, splitting by page ranges, compression for size reduction, and page-level editing. All operations are performed locally on the device, ensuring privacy and offline availability.

10.1 PDF Creation

PDF creation converts scanned images into multi-page PDF documents. The engine supports creating PDFs from a single image, multiple images (batch scan), or by importing existing files via the file_picker plugin. Each page is configured with A4 or Letter size, and images are scaled and centered with configurable margins. The PDF metadata includes title, author, creation date, and a unique document ID for tracking. OCR text is embedded as an invisible text layer behind the image, making the PDF searchable in standard PDF readers while preserving the visual fidelity of the original scan. Page numbering and custom watermarks can be optionally applied during creation.

10.2 Merge, Split & Compress Operations

Operation	Input	Output	Key Features
Merge	2+ PDF files	Single combined PDF	Custom page ordering, duplicate removal, bookmark merging
Split	Single PDF	Multiple PDFs	By page range, by bookmark, or individual pages
Compress	Single PDF	Smaller PDF	Image resampling, font subsetting, metadata stripping
Extract	Single PDF	Selected pages	Page range selection, rotation during extraction

10.3 Page Management

The PDF editor provides drag-and-drop page reordering, page rotation (90, 180, 270 degrees), page deletion, and page insertion from other PDFs or images. The editor uses a thumbnail-based interface where each page is represented by a preview image that can be manipulated via standard gesture controls. Changes are tracked in an edit stack that supports undo/redo operations. The editor maintains a working copy of the PDF in a temporary directory, only overwriting the original when the user explicitly saves. This non-destructive editing approach ensures that accidental edits can always be reverted, and the original document is never permanently altered without confirmation.

11. AI Engine Design

The AI Engine integrates Google Gemini (google_generative_ai: ^0.4.0) to provide intelligent document processing capabilities that go far beyond simple OCR. The engine operates on OCR-extracted text and document metadata to deliver features like summarization, key point extraction, smart renaming, auto-categorization, tag generation, translation, and structured data extraction. All AI features require an internet connection and are gated behind the premium subscription tier, though a limited number of free uses are provided for trial purposes.

11.1 Gemini API Integration

The Gemini API integration uses the Gemini 1.5 Flash model for fast, cost-effective inference on document processing tasks. The API client is configured with retry logic (3 attempts with exponential backoff), request timeout handling (30 seconds), and rate limiting to stay within quota. Each AI feature is implemented as a separate prompt template with carefully engineered system instructions that include few-shot examples for consistent output formatting. Responses are parsed using a structured output approach where the model returns JSON that is validated against expected schemas before being consumed by the application.

Feature	Description	Prompt Strategy
Document Summary	Generates a concise 2-3 sentence summary	Extractive then abstractive approach
Key Point Extraction	Lists 3-5 key points or action items	Structured bullet point output
Smart Rename	Suggests descriptive filename	Format: date_type_subject convention
Auto-Categorization	Assigns document to a category	Classification from predefined categories
Tag Generation	Creates 3-5 relevant tags	Controlled vocabulary with synonyms
Translation	Translates text to target language	Context-aware with domain terminology
Data Extraction	Extracts structured data from forms	JSON schema-driven extraction

12. Sync Engine Design

The Sync Engine implements an offline-first synchronization architecture using Firebase Firestore and Firebase Storage as the remote backend. The engine is designed to handle intermittent connectivity gracefully, queuing local changes for upload when network becomes available and resolving conflicts that arise from concurrent modifications on different devices. The sync process uses a delta-based approach that only transfers changed data, minimizing bandwidth usage and sync time.

12.1 Offline-First Pattern

The offline-first pattern ensures that all document operations (create, update, delete) work without network connectivity. Local changes are immediately persisted to Hive and a corresponding SyncRecord is created with pending status. A background sync worker (managed by workmanager: ^0.5.2) periodically checks for pending SyncRecords and attempts to push them to Firebase. Firestore's offline persistence is enabled for metadata, providing seamless access to previously synced document metadata even when offline. File uploads (images and PDFs) are queued and uploaded sequentially when connectivity is restored, with progress tracking and automatic resume on failure.

12.2 Conflict Resolution

Conflict resolution follows a "last write wins" strategy with a server-assigned timestamp as the tiebreaker. When a client pushes a change that conflicts with a newer server version, the sync engine compares timestamps and preserves the most recent modification. If the local change is older, it is discarded and the server version is applied locally. If the local change is newer, it overwrites the server version. In cases where data loss is possible (e.g., both versions have substantial changes), the engine creates a conflict entry in the SyncRecord with the conflicting server data preserved, allowing the user to manually resolve the conflict through the UI. This approach balances automatic resolution with user control over important data.

12.3 Delta Sync

Delta sync minimizes data transfer by only syncing records that have changed since the last successful sync. Each SyncRecord includes a timestamp, and the client tracks the lastSyncAt value in the UserProfile. During sync, the client queries Firestore for documents with updatedAt greater than lastSyncAt, downloads only the changed records, and applies them to the local Hive database. File-level delta sync uses hash comparison: if the file hash matches between local and remote, the file transfer is skipped entirely. This approach reduces typical sync operations from minutes to seconds, even for users with hundreds of documents.

13. Security Module Design

The Security Module provides multi-layered protection for user documents, combining device-level authentication, application-level access control, and data-level encryption. Security is treated as a first-class feature, not an afterthought, with every component designed to protect sensitive documents from unauthorized access. The module integrates with platform security APIs for biometric authentication and secure key storage, while providing user-configurable privacy controls that balance security with convenience.

13.1 Biometric Authentication

Biometric authentication uses the `local_auth` package (^2.2.0) to support fingerprint, face recognition, and iris scanning depending on device capabilities. The app can be configured to require biometric authentication on launch, when returning from background, or when accessing specific documents. A fallback PIN is required during biometric setup for devices where biometric hardware is unavailable or fails. The biometric check is implemented as a GoRouter redirect guard, intercepting navigation when authentication is required and presenting the system biometric prompt before allowing access. Failed attempts are tracked with exponential backoff, and after 5 failed attempts, a mandatory 30-second cooldown is enforced.

13.2 PIN Lock & Encryption

The PIN lock feature provides an alternative to biometric authentication, using a 4-6 digit PIN that is hashed using SHA-256 with a random salt and stored in `flutter_secure_storage` (^9.2.0). The secure storage uses Android Keystore on Android, ensuring that the PIN hash is protected by hardware-backed security. Document encryption uses AES-256-GCM via the `encrypt` package (^5.0.3), with encryption keys derived from the user's authentication credentials using PBKDF2 with 100,000 iterations. Encrypted documents are transparently decrypted on access and re-encrypted on save, with encryption applied at the file level so that individual documents can have different encryption keys. The encryption key management uses a master key stored in secure storage, with per-document keys encrypted under the master key.

13.3 Secure Storage & Privacy Controls

Secure storage is implemented using `flutter_secure_storage` for sensitive data (authentication tokens, encryption keys, PIN hashes) and Hive for non-sensitive data (preferences, UI state). Privacy controls allow users to configure which data is synced to the cloud, enable/disable OCR text storage, manage AI feature permissions, and delete all cloud data on demand. The app includes a privacy dashboard that shows what data is stored locally and remotely, with one-tap options to clear specific data categories. All network communication uses TLS 1.3, and API keys are stored in environment-specific configurations rather than hardcoded in the source code.

14. UI/UX Design System

ScanPro implements a comprehensive design system built on Material 3 (Material You), providing a consistent visual language across all screens and components. The design system defines color tokens, typography scale, spacing constants, elevation levels, and motion specifications that ensure visual coherence throughout the application. Every UI component is derived from the design system tokens, enabling theme switching (light/dark mode) and responsive layout adaptation with zero code duplication.

14.1 Material 3 Theming

The theme system uses Material 3's dynamic color scheme with a seed color derived from the ScanPro brand palette. The ThemeExtension API defines custom color tokens and component styles that extend the base Material 3 theme. Both light and dark color schemes are generated from the same seed color using the ColorScheme.fromSeed method, ensuring perceptual consistency across modes. The theme configuration includes custom shapes, elevation overlays, and component themes that override default Material 3 styles to match ScanPro's design language. Typography follows the Material 3 type scale with custom font families (Inter for UI, Times New Roman for document previews).

14.2 Dark/Light Mode & Responsive Layouts

Dark mode support is built into every component through the design system's token-based approach. Colors are never referenced directly; instead, semantic tokens (surface, onSurface, primary, etc.) resolve to different values based on the current brightness. The app follows the system brightness setting by default with an option to override. Responsive layouts use a breakpoint system: compact (phone, width < 600dp), medium (tablet portrait, 600-840dp), and expanded (tablet landscape, > 840dp). The scanner screen uses a different layout in expanded mode with a side panel for settings and recent scans, while the document list switches from a list view to a grid view at the medium breakpoint.

14.3 Animation System & Component Library

The animation system uses flutter_animate (^4.5.0) for declarative, chainable animations that are consistent and performant. Standard motion patterns include fade-in for screen transitions, slide-up for modal sheets, scale for card interactions, and shimmer for loading states. All animations respect the system accessibility setting for reduced motion, falling back to instant state changes when reduced motion is enabled. The component library provides 30+ custom widgets built on the design system tokens: DocumentCard, ScanButton, EnhancementChip, FolderTile, TagPill, SearchBar, ProgressIndicator, and more. Each widget includes light/dark variants, responsive sizing, and animation support, ensuring that new screens can be composed from existing components without inconsistency.

15. Dependencies List

ScanPro manages its dependencies through pubspec.yaml with strict version constraints using the caret syntax. The dependency list is organized by functional category and includes both production dependencies and dev dependencies for testing and code generation. All dependencies are evaluated for maintenance activity, license compatibility, and bundle size impact before inclusion. The application follows a policy of minimal dependencies, preferring platform APIs and custom implementations over third-party packages where feasible.

Package	Version	Purpose
flutter	sdk	Core Flutter framework
flutter_riverpod	^2.5.1	State management
riverpod_annotation	^2.3.5	Code generation for providers
go_router	^14.2.0	Declarative routing
hive	^2.2.3	Local NoSQL database
hive_flutter	^1.1.0	Hive Flutter integration
firebase_core	^3.0.0	Firebase initialization
firebase_auth	^5.0.0	Authentication service
cloud_firestore	^5.0.0	Cloud database
firebase_storage	^12.0.0	Cloud file storage
google_mlkit_text_recognition	^0.12.0	On-device OCR
google_mlkit_language_id	^0.11.0	Language detection
google_mlkit_translation	^0.11.0	On-device translation
syncfusion_flutter_pdf	^25.1.35	PDF creation and editing
syncfusion_flutter_pdfviewer	^25.1.35	PDF rendering and viewing
camera	^0.11.0	Camera hardware access
image_picker	^1.1.0	Gallery image selection
opencv_dart	^1.0.0	Computer vision processing
google_generative_ai	^0.4.0	Gemini AI integration
local_auth	^2.2.0	Biometric authentication
flutter_secure_storage	^9.2.0	Encrypted key-value storage

encrypt	^5.0.3	AES encryption
path_provider	^2.1.3	File system paths
uuid	^4.4.0	UUID generation
intl	^0.19.0	Internationalization and formatting
share_plus	^9.0.0	System sharing sheet
file_picker	^8.0.0	File selection from storage
permission_handler	^11.3.1	Runtime permission management
connectivity_plus	^6.0.3	Network connectivity detection
flutter_animate	^4.5.0	Declarative animations
cached_network_image	^3.3.1	Cached image loading
photo_manager	^3.2.0	Photo gallery access
mobile_scanner	^5.1.0	QR/barcode scanning
workmanager	^0.5.2	Background task scheduling
flutter_local_notifications	^17.1.0	Local notification display

16. Android Permissions

ScanPro requires specific Android permissions to access device hardware (camera, biometric sensors) and user data (photos, files, network). All permissions are declared in the AndroidManifest.xml file and are requested at runtime using the permission_handler package, following Android's permission best practices. Permissions are requested only when the user navigates to a feature that requires them, with clear explanations provided before each request. The application gracefully handles permission denials by offering alternative functionality or guiding users to settings.

Permission	Manifest Declaration	Purpose	Request Timing
Camera	CAMERA	Document scanning capture	On first scanner launch
Storage Read	READ_EXTERNAL_STORAGE	Import files from gallery	On gallery import
Storage Write	WRITE_EXTERNAL_STORAGE	Save scanned documents	On first save
Media Images	READ_MEDIA_IMAGES	Android 13+ image access	On gallery import

Internet	INTERNET	Cloud sync and AI features	Always available
Network State	ACCESS_NETWORK_STATE	Connectivity detection for sync	Always available
Biometric	USE_BIOMETRIC	Fingerprint and face unlock	On security setup
Vibrate	VIBRATE	Haptic feedback on capture	Always available
Foreground Service	FOREGROUND_SERVICE	Background sync operations	On sync enable
Post Notification	POST_NOTIFICATIONS	Sync completion alerts	On first sync
Boot Completed	RECEIVE_BOOT_COMPLETED	Restart sync after reboot	Always available

Additional AndroidManifest.xml configurations include: hardware acceleration enabled for smooth camera preview, large heap allocation for image processing, request for legacy external storage flag for compatibility, file provider declaration for sharing documents, and intent filters for deep link handling. The manifest also declares the camera hardware feature as required, which filters the app from devices without cameras on the Play Store.

17. Build Configuration

The build configuration manages compile-time settings, signing credentials, code optimization, and multi-flavor builds for different deployment environments. The configuration is split between the project-level build.gradle (Gradle wrapper version, repository sources, and plugin management) and the app-level build.gradle (SDK versions, signing, build types, and product flavors). Proper build configuration is essential for generating optimized, signed APKs and App Bundles for Play Store distribution.

17.1 build.gradle Configuration

Setting	Value	Notes
compileSdk	34	Latest Android SDK for API access
minSdk	24	Android 7.0+ (ML Kit requirement)
targetSdk	34	Latest Android behavior
kotlin version	1.9.22	Kotlin compiler version
NDK version	24.0.8215888	OpenCV native library support

Java compatibility	1.8	JDK 8 source/target compatibility
--------------------	-----	-----------------------------------

17.2 Signing Configuration & Flavors

The signing configuration uses separate keystores for debug and release builds. The debug keystore is checked into version control for team sharing, while the release keystore is stored in a CI/CD secure variable and never committed. The signing config references environment variables (KEYSTORE_PATH, KEYSTORE_PASSWORD, KEY_ALIAS, KEY_PASSWORD) that are set by the CI pipeline. Product flavors include "development" (with debug logging, API staging endpoints, and debug banner) and "production" (with minimal logging, production API endpoints, and no debug indicators). Each flavor has a distinct applicationIdSuffix and versionNameSuffix for side-by-side installation.

17.3 ProGuard Rules

ProGuard (R8) is enabled for release builds to reduce APK size and obfuscate code. Custom ProGuard rules preserve classes required by reflection-based libraries (Hive type adapters, Firebase models, Gson serialization). Specific keep rules are defined for: Hive adapter classes, Firebase data models, OpenCV native method signatures, and Syncfusion license validation classes. The ProGuard configuration is tested with each release build by running the full integration test suite on the obfuscated APK to catch any runtime class-not-found errors caused by over-aggressive shrinking.

18. MVP Roadmap

The MVP phase spans 8 weeks and focuses on delivering the core document scanning experience: capture, enhance, organize, and sync. The roadmap is structured in weekly sprints with clear deliverables at each milestone. The MVP must demonstrate that ScanPro can replace a physical scanner for the most common use cases while providing a polished, intuitive user experience that distinguishes it from competitors.

Week	Focus Area	Deliverables
Week 1-2	Project Setup & Foundation	Architecture scaffolding, theme system, navigation, CI/CD pipeline, lint rules, Hive initialization, Firebase project setup, base widget library
Week 3-4	Scanner Engine	Camera controller, real-time edge detection, auto-capture, perspective correction, image enhancement pipeline (auto/grayscale modes), gallery import, batch scan support
Week 5-6	Core Document Features	PDF creation from scans, OCR text extraction with ML Kit, document list/grid view, folder management, tagging system, search by title and OCR text, favorites and archive

Week 7-8	Cloud & Authentication	Firebase Auth (email/Google sign-in), Firestore sync for document metadata, Storage sync for files, offline-first pattern, sync status indicators, basic testing suite, bug fixes
----------	------------------------	---

At the end of the MVP phase, ScanPro will be a fully functional document scanner with cloud sync capabilities. Key success metrics for the MVP include: scan-to-PDF in under 10 seconds, OCR accuracy above 95% for English text, zero data loss during sync operations, and a crash-free rate above 99%. The MVP will be distributed to a closed beta group of 50 users for real-world testing and feedback collection before proceeding to the production roadmap phases.

19. Production Roadmap

The production roadmap extends ScanPro from an MVP scanner to a comprehensive document management platform with advanced features across three phases spanning months 3-12. Each phase builds on the previous one, adding capabilities that increase the application's value proposition and justify the premium subscription tier. The roadmap prioritizes features based on user feedback from the MVP beta, market analysis of competitor gaps, and technical dependencies between features.

19.1 Phase 2 (Month 3): PDF Tools & Search

Phase 2 focuses on transforming ScanPro from a scanner into a PDF toolkit. The PDF merge feature allows combining multiple scanned documents into a single file with custom page ordering. The PDF split feature extracts specific pages or page ranges from existing documents. PDF compression reduces file sizes by 40-70% through image resampling and font subsetting, making documents easier to share via email and messaging apps. Advanced OCR capabilities include multi-language recognition with automatic language detection, table structure recognition that preserves cell layouts, and handwriting recognition for annotated documents. The search engine gets a significant upgrade with full-text search across all OCR content, fuzzy matching for typo tolerance, and filter combinations by date range, folder, tags, and document type.

19.2 Phase 3 (Months 4-5): AI Features & Advanced Tools

Phase 3 introduces AI-powered features using the Gemini API integration. Document summarization generates concise overviews of lengthy documents, while key point extraction identifies the most important information. Smart rename automatically generates descriptive filenames based on document content, and auto-categorization assigns documents to appropriate folders. Tag generation creates relevant tags from document content, and translation supports cross-language document processing. The signature system allows users to create, save, and apply digital signatures to documents. The annotations module supports highlighting, text notes, freehand drawing, and text boxes on PDF pages. A QR/barcode scanner (mobile_scanner: ^5.1.0) is added as a supplementary feature, allowing users to scan and organize QR codes alongside traditional documents.

19.3 Phase 4 (Month 6+): Security & Enterprise

Phase 4 hardens the application for enterprise deployment and public release. The security module adds biometric authentication, PIN lock with configurable auto-lock timeout, AES-256 document encryption, and a privacy dashboard. Enterprise features include shared workspace folders with role-based access control, admin dashboards for team management, SSO integration via SAML/OIDC, and compliance reporting for regulated industries. Performance optimization targets include reducing app startup time to under 2 seconds, memory usage below 200MB during scanning, and battery consumption of less than 5% per hour of active scanning. The final step is Play Store preparation with store listing optimization, screenshot generation, privacy policy publication, and COPPA/GDPR compliance documentation. The target is Play Store launch by the end of month 8.

Phase	Timeline	Key Features	Success Metric
Phase 2	Month 3	PDF merge/split/compress, advanced OCR, search engine	50% faster document workflows
Phase 3	Months 4-5	AI features, signatures, annotations, QR scanner	30% premium conversion rate
Phase 4	Month 6+	Security module, enterprise features, Play Store launch	10K downloads in first month

20. Testing Strategy

ScanPro follows a comprehensive testing strategy that spans all levels of the testing pyramid: unit tests for business logic, widget tests for UI behavior, integration tests for end-to-end flows, and performance tests for critical paths. The testing infrastructure is built into the project from day one, not bolted on as an afterthought. Every feature module includes a corresponding test directory with test suites organized by architecture layer.

20.1 Unit Testing

Unit tests cover all domain use cases, repository implementations, and service classes. The test framework uses Flutter's built-in test package with mockito for generating mocks and faker for creating test data. Each use case has at least 3 test cases: happy path, error handling, and edge cases. Repository tests verify data mapping between models and entities, cache invalidation logic, and error propagation. Service tests use mock implementations of platform APIs (camera, ML Kit, Firebase) to verify business logic in isolation. The target is 80% code coverage for the domain layer and 70% for the data layer. Tests run automatically on every pull request via GitHub Actions.

20.2 Widget Testing

Widget tests verify UI behavior including rendering, user interactions, state transitions, and navigation. Each screen has a dedicated widget test file that tests the complete user interaction flow: initial rendering state, loading states, data states, and error states. Tests use Riverpod overrides to inject mock providers, ensuring tests are deterministic and fast. Key scenarios tested include: document list pagination, scanner capture flow, OCR result display, PDF viewer interaction, and settings persistence. Widget tests also verify accessibility by checking semantic labels, touch target sizes, and contrast ratios. The target is 60% widget code coverage.

20.3 Integration & Performance Testing

Integration tests verify complete user flows across multiple screens and system components using Flutter's `integration_test` package. Key integration test scenarios include: full scan-to-sync workflow (capture, enhance, create PDF, upload to cloud), authentication and sync across app restarts, and document sharing via system intents. Integration tests run on real Android devices connected via USB, using Firebase emulator for backend services. Performance testing targets specific metrics: camera preview frame rate (target 30fps), OCR processing time (under 3 seconds per page), PDF generation speed (under 5 seconds for 10 pages), and memory usage during batch scanning (under 300MB). Performance benchmarks are tracked across releases to detect regressions, with alerts triggered when metrics degrade by more than 15% from the baseline.

Test Type	Coverage Target	Tools	Run Frequency
Unit Tests	80% domain, 70% data	flutter test, mockito, faker	Every PR
Widget Tests	60% presentation	flutter test, riverpod overrides	Every PR
Integration Tests	5 critical flows	integration_test, Firebase emulator	Nightly
Performance Tests	4 key metrics	flutter drive, custom metrics	Weekly